

Final Project Report: Student Management System

Course / Track: 6-Week Java Development Internship
Week: Week 6 Final Project
Project Title: Console-Based Student Management System in Java
Repository: <https://github.com/snehakesharwani05/student-management-system>

1. Project Title

Student Management System (Java Console Application)

2. Introduction

The Student Management System is a modular, console-driven software application developed using Core Java. Designed for academic institutions and training programs, the system manages the lifecycle of student records—encompassing registration, updates to enrollment details and grades, search queries, and record deletion. It combines in-memory data structures for fast lookups with persistent file storage and an asynchronous background daemon thread to eliminate the risk of data loss upon unexpected system termination.

3. Objective

- Apply and integrate core Object-Oriented Programming (OOP) paradigms (Encapsulation, Inheritance, Polymorphism, Abstraction) into a production-style architecture.
- Implement end-to-end CRUD (Create, Read, Update, Delete) operations with defensive input validation.
- Utilize the Java Collections Framework (specifically HashMap and ArrayList) to balance rapid retrieval and structured sequential iteration.
- Implement persistent file storage using standard Java character streams (BufferedReader and BufferedWriter).
- Incorporate multithreading via a background auto-save daemon that continuously flushes memory state to disk.
- Maintain version control, issue tracking, and clean documentation using Git and GitHub.

4. Features

Feature	Description	Technical Highlight
Add Student Record	Captures and validates student ID, full name, age,	Stores entries directly into an in-memory HashMap and

Feature	Description	Technical Highlight
	course name, and academic marks.	tracks total instances using a static counter.
View All Records	Displays formatted tabular listings of all registered students.	Converts values to an ArrayList and calls overridden polymorphic string formatters.
Search Record	Retrieves detailed student information instantly by inputting a unique Student ID.	Executes in $O(1)$ constant time; throws a custom StudentNotFoundException if absent.
Update Record	Modifies course enrollment and academic marks for existing student records.	Validates presence prior to mutation and updates state safely in memory.
Delete Record	Purges a student record permanently from memory and disk.	Removes key-value pairs with thread-safe synchronized locks.
Asynchronous Auto-Save	Automatically writes in-memory records to data/students.txt every 30 seconds.	Executes in a background Runnable daemon thread without interrupting user console workflows.
Defensive Input Handling	Prevents crashes caused by datatype mismatches (e.g., entering letters for numeric IDs).	Uses comprehensive try-catch blocks to catch and clear erroneous input streams.

5. Concepts Used

- **Object-Oriented Programming (OOP):**
 - **Abstraction:** Defined an abstract class Person with core attributes and an abstract method getDetails().
 - **Inheritance:** Student extends Person, reusing identity fields while adding specialized fields like course and marks.
 - **Encapsulation:** Private member variables accessed solely through public getters and setters with state validation.
 - **Polymorphism:** Overridden getDetails() method provides dynamic runtime representation.
 - **Keywords:** Proper use of this, super() constructor chaining, and static variables.
- **Interfaces & Loose Coupling:** The IStudentService interface establishes a strict

operational contract between user interactions and data handling layers.

- **Collections Framework:** Utilized `HashMap<Integer, Student>` for $O(1)$ searches, updates, and deletes, paired with `ArrayList` for ordered output generation.
- **Exception Handling:** Created custom checked exception `StudentNotFoundException` alongside robust handling of `InputMismatchException` and `IOException`.
- **File Handling (I/O):** Utilized `BufferedReader` and `BufferedWriter` with comma-separated (CSV) parsing to achieve non-volatile persistence.
- **Multithreading:** Implemented the `Runnable` interface in `AutoSaveThread` running as a daemon with synchronized locks to eliminate race conditions.
- **Version Control:** Managed project history using Git commits and GitHub repository synchronization.

6. Project Structure

```
student-management-system/  
├── .gitignore  
├── README.md  
├── data/  
│   └── students.txt  
├── screenshots/  
│   ├── add_student.png  
│   ├── view_students.png  
│   └── search_update.png  
└── src/  
    ├── Main.java  
    ├── exception/  
    │   └── StudentNotFoundException.java  
    ├── model/  
    │   ├── Person.java  
    │   └── Student.java  
    └── service/  
        ├── AutoSaveThread.java  
        ├── IStudentService.java  
        └── StudentServiceImpl.java
```

7. Code Overview

Below are the foundational segments of the source code demonstrating the implementation:

7.1 Service Implementation & Thread Safety

```
public synchronized void addStudent(Student student) {  
    studentMap.put(student.getId(), student);  
}
```

```

public synchronized Student searchStudentById(int id) throws StudentNotFoundException {
    if (!studentMap.containsKey(id)) {
        throw new StudentNotFoundException("Student with ID " + id + " not found.");
    }
    return studentMap.get(id);
}

```

7.2 Background Multithreading Daemon

```

public class AutoSaveThread implements Runnable {
    private final IStudentService service;
    private volatile boolean running = true;

```

```

    public AutoSaveThread(IStudentService service) {
        this.service = service;
    }

```

```

    @Override
    public void run() {
        while (running) {
            try {
                Thread.sleep(30000); // Snapshot saved every 30 seconds
                service.saveToFile();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                break;
            }
        }
    }
}

```

8. Output & Execution Flow

===== STUDENT MANAGEMENT SYSTEM =====

```

1. Add Student Record
2. View All Records
3. Search Student by ID
4. Update Student Details
5. Delete Student Record
6. Save & Exit
Enter your choice (1-6): 1
Enter Student ID (Integer): 101
Enter Name: Sneha Kesharwani
Enter Age: 20

```

Enter Course: B.Tech CSE
Enter Marks: 92.5
Student record added successfully!

===== STUDENT MANAGEMENT SYSTEM =====
Enter your choice (1-6): 2

--- Registered Students ---
ID: 101 | Name: Sneha Kesharwani | Age: 20 | Course: B.Tech CSE | Marks: 92.50

9. Challenges Faced & Solutions

Challenge	Root Cause	Resolution
Scanner Buffer Skipping	Calling scanner.nextLine() after numeric reads (nextInt()/nextDouble()) captured dangling newline characters instead of waiting for user text.	Added explicit newline clearance calls (scanner.nextLine();) immediately following all numeric parsing.
Thread Race Conditions	The background auto-saver thread wrote map values to disk while the main CLI thread was simultaneously inserting or modifying entries.	Synchronized shared data mutations in StudentServiceImpl using synchronized methods to enforce mutual exclusion.
First-Run File Errors	Application threw FileNotFoundException when launched in a fresh environment before the data/ folder or file was created.	Added directory and file existence checks with file.exists() and directory.mkdir() prior to initiating file read/write streams.
Git Remote Collision	Configured Git remote pointed to a non-existent placeholder path, blocking code synchronization.	Re-pointed the remote URL using git remote set-url origin directly to the target GitHub repository and re-authenticated credentials.

10. Conclusion

The development of the Student Management System successfully synthesized theoretical Java concepts learned across the 6-week internship into an industry-standard, working software architecture. Key learnings include decoupling application interfaces from underlying implementations, managing concurrent thread states safely, parsing formatted data streams for

persistence, and adopting standard Git version control practices. The project provides an extensible foundation capable of being integrated with relational databases and graphical desktop interfaces in future iterations.